

HARMONY: A Scalable Distributed Vector Database for High-Throughput Approximate Nearest Neighbor Search

QIAN XU, Renmin University of China, China
FENG ZHANG*, Renmin University of China, China
CHENGXI LI, Renmin University of China, China
LEI CAO, Massachusetts Institute of Technology, USA
ZHENG CHEN, Renmin University of China, China
JIDONG ZHAI, Tsinghua University, China
XIAOYONG DU, Renmin University of China, China

Approximate Nearest Neighbor Search (ANNS) is essential for various data-intensive applications, including recommendation systems, image retrieval, and machine learning. Scaling ANNS to handle billions of high-dimensional vectors on a single machine presents significant challenges in memory capacity and processing efficiency. To address these challenges, distributed vector databases leverage multiple nodes for the parallel storage and processing of vectors. However, existing solutions often suffer from load imbalance and high communication overhead, primarily due to traditional partition strategies that fail to effectively distribute the workload. In this paper, we introduce Harmony, a distributed ANNS system that employs a novel multi-granularity partition strategy, combining dimension-based and vector-based partition. This strategy ensures a balanced distribution of computational load across all nodes while effectively minimizing communication costs. Furthermore, Harmony incorporates an early-stop pruning mechanism that leverages the monotonicity of distance computations in dimension-based partition, resulting in significant reductions in both computational and communication overhead. We conducted extensive experiments on diverse real-world datasets, demonstrating that Harmony outperforms leading distributed vector databases, achieving $4.63\times$ throughput on average in four nodes and 58% performance improvement over traditional distribution for skewed workloads.

CCS Concepts: • **Computer systems organization** → **Distributed architectures**.

Additional Key Words and Phrases: Vector Database, Distributed Database, Dimension-based Pruning

ACM Reference Format:

Qian Xu, Feng Zhang, Chengxi Li, Lei Cao, Zheng Chen, Jidong Zhai, and Xiaoyong Du. 2025. HARMONY: A Scalable Distributed Vector Database for High-Throughput Approximate Nearest Neighbor Search. *Proc. ACM Manag. Data* 3, 4 (SIGMOD), Article 249 (September 2025), 28 pages. <https://doi.org/10.1145/3749167>

*Feng Zhang is the corresponding author of this paper.

Authors' Contact Information: Qian Xu, Renmin University of China, China, xuqianmamba@gmail.com; Feng Zhang, Renmin University of China, China, fengzhang@ruc.edu.cn; Chengxi Li, Renmin University of China, China, 1633384450@qq.com; Lei Cao, Massachusetts Institute of Technology, USA, lcao@csail.mit.edu; Zheng Chen, Renmin University of China, China, chenzheng123@ruc.edu.cn; Jidong Zhai, Tsinghua University, China, zhaijidong@tsinghua.edu.cn; Xiaoyong Du, Renmin University of China, China, duyong@ruc.edu.cn.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 2836-6573/2025/9-ART249
<https://doi.org/10.1145/3749167>

1 Introduction

Approximate nearest neighbor search (ANNS) is at the core of many data-intensive applications such as search engines [17, 42, 46], e-commerce platforms [44, 76], recommendation systems [40, 43, 47, 68] and natural language processing tasks [49, 52, 56]. Despite significant advancements in indexing structures and query algorithms [29, 51], over 90% of the time spent on ANNS is dedicated to computing distances between vectors in cluster-based indices [29]. Moreover, the volume of unstructured data, which often must be converted to high dimensional vectors before analysis, continues to increase. For instance, during Alibaba's shopping festival, 500PB of data is generated daily [27], while YouTube receives over 500 hours of content every minute [3]. This escalating demand highlights the need for distributed solutions to store vectors and process ANNS queries in parallel across multiple nodes. Distributed computing has achieved remarkable success across various domains, including web-scale indexing [22] and big data analytics [66, 72]. Building on these successes, distributed vector databases [62, 79] have emerged as a promising approach to accelerate large-scale ANNS. However, the high dimensionality of vectors and the need for full-dimensional distance computations make conventional partitioning methods unsuitable. Moreover, the popular graph-based segmentation in standalone machines is not well compatible with distributed features, as query paths for vectors tend to introduce edges across machines, resulting in high latency. Thus, our work targets distributed-friendly vector placement and optimized distance computation in cluster-based indexes.

Distributed vector databases provide benefits in both space efficiency and time overhead reduction. Existing studies show that most time in Approximate Nearest Neighbor Search (ANNS) is spent on distance calculations between vectors. In cluster-based vectors, distance computations accounted for 99.7% of the total time when conducting searches with precision 99% on the *msong* dataset [29]. Optimizing this time overhead can significantly enhance ANNS speed. Distributed vector databases can substantially reduce query latency by processing large ANNS workloads concurrently [62]. On the other hand, distributing vectors across multiple machines alleviates memory pressure on individual nodes. These advantages make distributed architectures an ideal solution for handling the increasing volume of vector data in modern applications.

Despite advancements, distributed acceleration for Approximate Nearest Neighbor Search (ANNS) still faces three key challenges. First, traditional partition methods, such as cluster-based or hash-based partition, often result in load imbalances in ANNS systems. These imbalances occur when certain machines handle disproportionately large computations, which degrade performance. An adaptive indexing scheme is required to resolve this issue and enhance ANNS efficiency. Second, distributing queries across machines while minimizing communication and computation overhead remains challenging in distributed ANNS systems. Distributed vector databases often transmit intermediate results, which typically account for less than 10% of the size of the original vectors [12, 79]. Although smaller, the communication overhead remains significant due to the bandwidth disparity between transmission (up to 100Gb/s) and computation (hundreds of GB/s), causing delays [12]. Simple query distribution strategies can cause bottlenecks and degrade performance, especially under imbalanced workloads. Third, inter-machine pruning offers opportunities to reduce redundant computations in ANNS, but designing lightweight pipelines for pruning with minimal overhead is a major challenge in improving query efficiency.

Previous research on distributed vector databases has focused on two main aspects: (1) improving query performance in multi-machine settings [8, 12, 13], and (2) ensuring the accuracy of query results across multiple nodes [10, 32]. However, these studies do not address performance guarantees under load imbalances or explore pruning strategies that reduce inter-machine workload. In addition

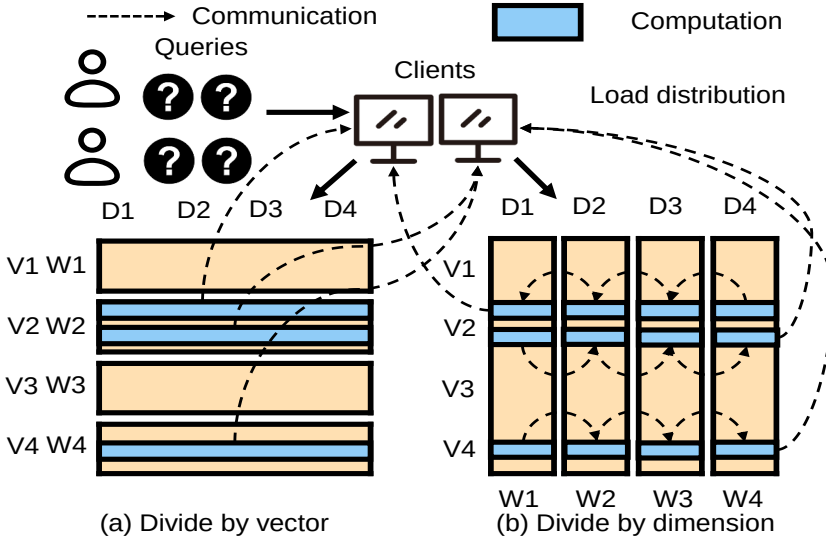


Fig. 1. An example of overhead in different partition granularity when searching. Where V_i denotes the i th block according to vector-based partition and D_j denotes the j th block according to dimension-based partition.

to coordinating machines, inter-machine pruning can also reduce workload and balance imbalanced loads, ultimately improving throughput.

We have made two key observations. First, vectors can be partitioned at various granularities, such as vector-based and dimension-based partition, as shown in Figure 1. Each strategy has distinct computational and transmission loads, with $W1-W4$ representing different computing machines. Specifically, traditional vector-based partition [62, 79] involves dividing the dataset by assigning entire vectors to different nodes ($V1-V4$ in Figure 1 (a)). This approach simplifies distance computation, with each node communicating with the master node only before and after the calculation. However, it often causes load imbalance, with some nodes becoming overloaded with “hot” partitions, while others remain underutilized, particularly under uneven query workloads. For example, as shown in Figure 1 (a), in the case of an unbalanced load, only $W2$ and $W4$ perform the distance computation. In contrast, as shown in Figure 1 (b), dimension-based partition distributes the dataset by splitting the dimensional space across nodes, with each node handling a subset of the vector dimensions ($D1-D4$ in Figure 1 (b)). This method ensures a balanced load across all nodes regardless of the query patterns, as each node contributes equally to the distance computations. However, it necessitates multiple communications between the client and the nodes for each query, increasing the communication overhead and potentially impacting query latency.

Second, we find that dimension-based partition allows early stopping during distance computations. For instance, in Figure 1(b), if the partial distance computed from $W1$ and $W2$ exceeds the current worst distance, the remaining computations in dimensions $W3$ and $W4$ can be safely skipped. This early termination enables pipelined processing and pruning techniques, significantly reducing computational burden and accelerating query responses.

Based on these observations, we present Harmony, a high throughput and robust multi-node optimized nearest neighbor search system. It introduces three main innovations. First, Harmony integrates vector-based and dimension-based partition into a hybrid index construction framework.

This approach, guided by a cost model, dynamically adapts to the characteristics of each query and the current cluster state, ensuring efficient query processing and stable performance under various loads. Second, Harmony addresses load imbalance by combining dimension-based and vector-based partition in a hybrid granularity scheme, ensuring efficient load distribution and maintaining high throughput for balanced workloads. Third, Harmony introduces a pruning technique based on the monotonicity of distance computations in dimension-based partition. When partial vector dimensions no longer contribute to the final result, redundant calculations are skipped. This early pruning, coupled with a pipelined execution model, minimizes unnecessary computations and communications, especially for high-dimensional queries.

Our experimental evaluation on multiple real-world datasets shows that Harmony achieves substantial performance gains over state-of-the-art vector databases, with a 4.63× throughput improvement on average in four nodes and a 58% improvement over traditional distribution for skewed loads. We summarize our main contributions as follows:

- We observe that dimension-based partition enables an even distribution of computational demands. Integrating it with vector-based partition, we propose a multi-granularity approach that optimizes resource allocation across nodes, improving system efficiency.
- We find that dimension-based partition allows early stopping during distance computations. We design a pruning strategy that reduces computational and communication overhead by utilizing intermediate results at the dimension level across nodes.
- We conduct comprehensive experiments demonstrating that, compared to the open-source system Faiss, Harmony achieves a 4.63× throughput improvement on average in four-node.

2 Background and Related Work

2.1 Vector Database Systems

With the rise of deep learning, vectors are now integral to a wide range of applications, including search engines [17, 42, 46], e-commerce platforms [44], recommendation systems [40, 43, 47, 68], and natural language processing tasks [52, 56, 69, 74, 75]. Vectors serve as representations for diverse data types, such as videos, texts, images, and audio. To enhance the performance of these applications, efficient vector search becomes crucial, driving the development of optimized vector databases [26, 28, 30, 41, 45, 51, 55, 65, 67, 70, 71, 73, 77].

Vector databases face considerable challenges due to the rapid expansion of vector data. For example, the UCI Machine Learning Repository [2] lists over 100 open-source datasets with dimensions exceeding 100. At the same time, the "online" nature of these systems [21, 31, 35, 54] demands that vector searches be completed within milliseconds. This strict latency requirement contrasts with the exact search algorithms used in relational databases [18], where most vectors must be scanned to retrieve the necessary results.

To meet these performance demands, vector database systems employ indexes to efficiently locate vectors. These indexes generally fall into two categories: graph-based and partition-based. Graph-based indexes [23, 24, 26, 27, 33, 51, 59, 60, 77] construct a graph based on vector distances, where the query process involves a breadth-first search. Partition-based indexes, such as cluster-based [14, 15, 17, 25, 37–39, 71, 78], hash-based [20, 34, 57, 63, 64], and high-dimensional tree-based [16, 48, 53, 61] methods, divide vectors into partitions based on distance metrics, enabling faster identification of nearest clusters before searching within them.

Although these indexing techniques improve query speed, they incur additional storage overhead. Since full-dimensionality is necessary to compute vector distances accurately, reducing storage costs without resorting to lossy compression techniques such as quantization [25] remains a challenge. As a result, attention is shifting towards distributed vector Approximate Nearest Neighbor Search (ANNS) schemes to address these challenges.

2.2 Distributed Solutions in Data Management

Beyond specialized vector databases, numerous distributed data management systems in both graph and relational domains offer valuable insights into handling massive datasets. For instance, graph databases like Neo4j [7], TigerGraph [11], and JanusGraph [1] employ partitioning (or sharding) strategies that split large graphs across multiple machines, aiming to reduce cross-partition edges and data transfers during queries. Similarly, distributed relational databases such as Google Spanner [19], CockroachDB [5], and F1 [58] achieve horizontal scalability by sharding tables across commodity servers, enabling transactional consistency and parallel query execution. Distributed query engines like Presto [9], Spark SQL [6], and Apache Drill [4] apply massive parallel processing (MPP) and in-memory computation to further reduce query latencies.

Although these systems primarily target structured or graph-shaped data, they share common design principles that are highly relevant to vector databases. First, they demonstrate the importance of robust data partitioning, where carefully chosen partitioning schemes minimize inter-node communication and balance workloads. Second, they underline the critical role of parallel processing frameworks in reducing query latency and scaling to large datasets. Third, they highlight potential challenges such as load imbalance and excessive data transfers, which can degrade performance if not properly managed. As high-dimensional vector data continues to grow in volume and complexity, leveraging these established techniques—while also addressing the unique aspects of vector data—presents an opportunity to build more efficient and scalable distributed vector database solutions [8, 10, 12, 13, 50, 79].

3 Motivation

3.1 Revisiting Search and Cost Distribution in Distributed Vector Databases

Motivation 1: Dimension-level pruning. Dimension-level pruning has been shown to be highly effective for accelerating vector search in single-machine settings [29, 41]. It exploits the additive nature of common distance functions by decomposing the total distance into partial results over disjoint subsets of vector dimensions.

For example, in the case of squared Euclidean distance, given disjoint index sets $I_1, I_2, \dots, I_M$ partitioning the dimensions, the full distance between vectors $\mathbf{p}, \mathbf{q} \in \mathbb{R}^d$ can be written as:

$$D^2(\mathbf{p}, \mathbf{q}) = \sum_{k=1}^M d_k^2(\mathbf{p}, \mathbf{q}), \quad \text{where} \quad d_k^2(\mathbf{p}, \mathbf{q}) = \sum_{i \in I_k} (p_i - q_i)^2.$$

This structure allows the system to prune a candidate $\mathbf{q}$ early if the cumulative partial distance $S_k^2 = \sum_{j=1}^k d_j^2(\mathbf{p}, \mathbf{q})$ already exceeds the best-so-far threshold τ^2 , since remaining terms are non-negative.

Empirical effectiveness. We conduct a simple experiment to validate its pruning power. Using four groups of dimensions (each covering one-quarter of the full vector), we observe:

- Nearly 50% of candidate vectors can be pruned after only two groups.
- Over 80% are pruned by the time the third and fourth groups complete, reaching up to 97.4% in our setting.

These results confirm that dimension-level pruning can eliminate the majority of distance computations when applied properly.

Motivation 2: Cost breakdown and skewed loads effects in distributed vector databases.

Cost breakdown. We evaluate two commonly used partitioning strategies—dimension-based (D) and vector-based (V)—under both blocking (B) and non-blocking (NB) communication modes. Blocking communication requires a machine to wait for data from other nodes before proceeding

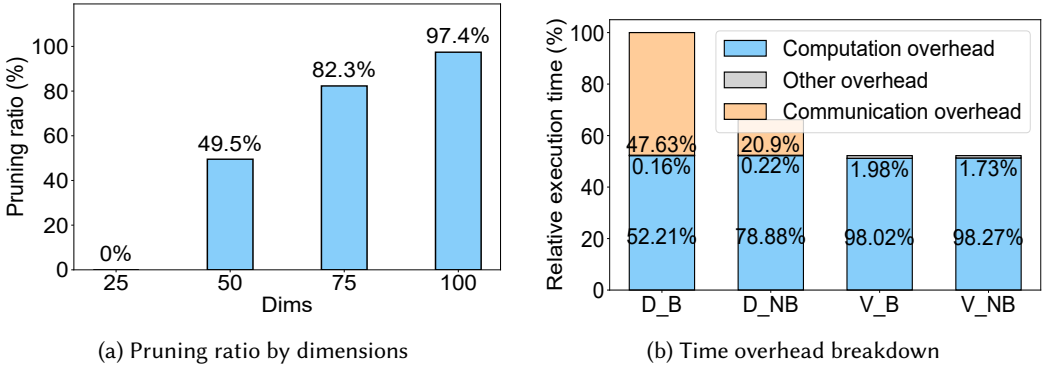


Fig. 2. Pruning ratio and time overhead breakdown.

with its computation. This is typical in dimension-based partitioning, where each node processes a subset of vector dimensions and must receive partial results from others to compute full distances. In contrast, non-blocking communication allows a machine to overlap data transfer with local computation, improving resource utilization. Experiments are conducted on the Sift1M dataset using a five-node cluster (one client and four workers), each equipped with 56 vCPUs and connected via 100 Gb/s links. Figure 2(b) presents the relative execution time for each setting, with computation (blue), communication (orange), and other overheads (gray) shown. The total height of each bar indicates the total execution time, and each component is labeled with its percentage.

We observe that vector-based partitioning incurs negligible communication overhead under both blocking and non-blocking modes. In contrast, dimension-based partitioning benefits significantly from non-blocking communication, with an average reduction of about 66% in communication time compared to the blocking mode. This is attributed to the frequent exchange and synchronization of intermediate results required by dimension-based strategies. The computation overheads are similar across all four settings, with vector-based partitioning being slightly more efficient due to reduced thread switching and more complete SIMD (e.g., AVX) execution. However, this advantage is minimal, and the overall performance difference is primarily driven by the communication pattern.

Stability. We observe a stark contrast in query bandwidth stability under balanced and imbalanced workloads between the two partitioning approaches. Specifically, dimension-based partitioning delivers a consistently stable query bandwidth, regardless of whether the load is evenly distributed across nodes. In contrast, vector-based partitioning achieves 30% higher bandwidth than dimension-based partitioning under balanced loads, but its performance degrades substantially under load imbalance. With skewed workloads in a five-node setup, vector-based partitioning can drop to as low as 26% of the cluster-based approach’s bandwidth, underscoring the trade-off between maximizing throughput under balanced conditions and maintaining stability when workloads vary.

3.2 Challenges

Distributed vector databases promise scalable and efficient approximate nearest neighbor search (ANNS), but three key challenges must be addressed to realize their potential.

Challenge 1: Combining partitioning strategies in ANNS. A key challenge is designing an indexing scheme that integrates both vector-based and dimension-based partitioning effectively. Vector-based partitioning reduces communication overhead, while dimension-based partitioning balances workloads. However, merging these strategies requires adaptive mechanisms to exploit their strengths under varying query distributions. Achieving high throughput and stable performance across diverse workloads requires dynamic partitioning policies and runtime optimizations.

Challenge 2: Efficient query distribution in ANNS. Even with robust partitioning, queries must be efficiently distributed across machines to minimize communication and computation overhead while ensuring load balancing. Merely splitting queries among nodes does not guarantee the avoidance of bottlenecks. Consequently, innovative query assignment and workload distribution techniques are necessary to allocate queries in a manner that reduces network traffic, prevents node saturation, and maintains low latency. These mechanisms should also adapt to shifting workloads and handle skewed query distributions seamlessly.

Challenge 3: Effective pruning in distributed environments. While dimension-level pruning shows strong potential, it becomes less effective in distributed settings due to synchronization overhead. Since pruning decisions require partial results from multiple nodes, the system is bottlenecked by the slowest participant. For instance, if four nodes achieve 10%, 20%, 30%, and 40% pruning respectively, the overall speedup is only 10%, as all nodes must compute before a candidate can be pruned. This highlights the need for distributed-friendly pruning strategies that minimize coordination and enable early termination with low communication cost.

4 System Design

4.1 Overview

We introduce Harmony, a cluster-based distributed ANNS engine that can address the limitations of existing distributed partitioning strategies by integrating multi-granularity partitioning and multi-node friendly dimension-level pruning.

Solution to challenges. Harmony tackles the three major challenges described in Section 3.2 through the following key ideas.

1) *Adaptive partitioning strategy for optimal throughput and stability.* To combine vector-based and dimension-based partitioning, Harmony adopts a dynamic approach that uses a cost model to determine the most efficient partition strategy based on the current load distribution. The system balances vector-based partitioning, which reduces communication overhead, with dimension-based partitioning, which ensures workload balance. This adaptive scheme enhances throughput and stability across diverse query workloads, optimizing the benefits of both approaches.

2) *Efficient query distribution with load-aware routing.* For efficient query distribution, Harmony implements a load-aware routing mechanism that minimizes communication overhead while ensuring balanced query processing. The system adjusts query assignments based on real-time workload analysis, ensuring nodes are neither overwhelmed nor underutilized. By distributing queries to minimize communication overhead and prevent node saturation, Harmony ensures that query execution is both fast and balanced, even under skewed query distributions.

3) *Multi-machine pruning with lightweight pipelines.* To tackle the challenge of multi-machine pruning, Harmony introduces a lightweight inter-machine pruning mechanism that propagates partial results and prunes irrelevant computations across nodes. We design a pruning strategy that is well-suited for distributed environments, enabling efficient query execution through early elimination of unnecessary computations. By synchronizing nodes with minimal communication overhead, Harmony reduces redundancy and fully exploits parallelism in large-scale ANNS deployments.

Models. Harmony is designed with two key modules: ❶ fine-grained query planner (detailed in Section 4.2) and ❷ flexible pipelined execution engine (detailed in Section 4.3). Figure 3 illustrates the overall workflow and how each module addresses the primary challenges.

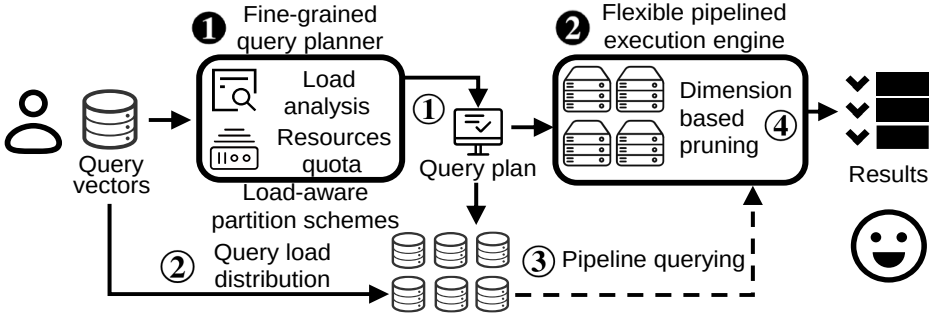


Fig. 3. Harmony's query process.

- **① Load-aware query plan** and **② Query load distribution**. These components analyze resource usage and query skewness, leveraging a cost model to select between dimension-based, vector-based, or hybrid partitioning. Building on the chosen partition strategy, Harmony distributes query vectors across nodes according to the anticipated workload, ensuring balanced resource utilization.
- **③ Pipeline querying** and **④ Dimension-based pruning**. To avoid parallel processing of all vector dimensions at each node, these components adopt a pipeline-based flow. Partial results are passed sequentially along nodes to avoid duplicative work on candidates with pruning potential. As partial results accumulate, vectors exceeding the pruning threshold are discarded before reaching other nodes.

These components work together to adaptively partition data based on the current workload and exploit dimension-level pruning, ensuring robust performance and reduced query latencies across diverse high-dimensional scenarios.

Novelties. Harmony offers three key innovations:

- **Adaptive index construction.** Harmony adopts an adaptive indexing approach that dynamically adjusts the index structure based on workload characteristics. This ensures efficient handling of varying query types and workloads, maintaining high performance across scenarios.
- **Load-aware partitioning for balanced distribution.** By integrating dimension-based and vector-based partitioning, Harmony ensures that vectors are distributed across nodes to minimize load imbalance. This ensures stable performance under skewed loads while preserving high throughput for balanced workloads.
- **Dimension-level pruning.** Harmony incorporates dimension-level pruning in its query processing pipeline. By executing partial distance calculations across nodes, vectors that cannot exceed the current best threshold are pruned early, reducing unnecessary computation and communication, especially for high-dimensional queries.

4.2 Fine-grained Query Planner

To achieve optimal partitioning and query strategy, we introduce a cost model that evaluates computational and communication overhead during execution. Table 1 shows the key variables and metrics used in the cost model. Like traditional databases, computational and transmission overheads can be efficiently estimated during the initial query setup. For instance, lightweight metrics such as the number of cluster centers in distance calculations or the dimensionality of query vectors allow the cost model to be computed with minimal overhead.

4.2.1 Cost model. Let $Q = \{q_1, q_2, \dots, q_{|Q|}\}$ be the set of query vectors and N the number of nodes.

Table 1. Notations for Our Cost Model

Symbol	Meaning
π	Partition plan
$\mathcal{B}_{\text{dim}}(\pi), \mathcal{B}_{\text{vec}}(\pi)$	Dimension- and vector-based partitions in π
d, v	Specific dimensions and vectors, respectively
$n(d, v)$	Node n with dimension d and vector v in π
$c_{\text{comp}}(d, v, q)$ $c_{\text{comm}}(d, v, q)$	Computation/communication costs for query q when processed in block (d, v)
$\text{Load}(d, v, \pi)$	Total load on node (d, v) under plan π
$I(\pi)$	Imbalance factor (variance)
α	Weight for imbalance in overall cost

Query cost. For each query $q \in Q$, we consider the joint processing over all pairs of dimension and vector partitions defined by the plan π . The per-query cost is:

$$\begin{aligned}
C_q(\pi) = & \underbrace{\sum_{d \in \mathcal{B}_{\text{dim}}(\pi)} \sum_{v \in \mathcal{B}_{\text{vec}}(\pi)} c_{\text{comp}}(d, v, q)}_{\text{Computation cost (Load)}} \\
& + \underbrace{\sum_{d \in \mathcal{B}_{\text{dim}}(\pi)} \sum_{v \in \mathcal{B}_{\text{vec}}(\pi)} c_{\text{comm}}(d, v, q)}_{\text{Communication cost}}
\end{aligned}$$

Imbalance factor. We define $\text{Load}(n, \pi)$ as the total computation cost assigned to $n(d, v)$. For each node n , the load is computed by summing up $c_{\text{comp}}(d, v, q)$ over all queries q assigned to n .

The *imbalance factor* $I(\pi)$, also called variance, is defined via the standard deviation:

$$I(\pi) = \sqrt{\frac{1}{N} \sum_{n=1}^N [\text{Load}(n, \pi) - \bar{L}]^2}, \text{ where } \bar{L} = \frac{1}{N} \sum_{n=1}^N \text{Load}(n, \pi).$$

A larger imbalance implies that certain nodes may overload.

Overall cost function. We combine the per-query costs and imbalance term into a single end-to-end objective:

$$C(\pi, Q) = \sum_{q \in Q} C_q(\pi) + \alpha \times I(\pi)$$

Here, α is a learned weight that reflects how much system performance is affected by load imbalance, given the current hardware configuration. Unlike fixed penalties, Harmony adapts α based on offline profiling across heterogeneous cluster setups. For example, if one of the nodes is significantly slower than others, α will increase to penalize load skew more heavily, capturing its impact on overall query latency.

Example application. Figure 4 (a) illustrates how Harmony uses the cost model to change partition strategies. In initial plan π where $\mathcal{B}_{\text{dim}}(\pi) = 6$ and $\mathcal{B}_{\text{vec}}(\pi) = 1$, the average computation and communication costs per query are $c_{\text{comp}} = 80$ ms and $c_{\text{comm}} = 40$ ms, respectively. The imbalance factor $I(\pi)$ is 0, and the learned α based on hardware profiling is 0.3 ms, leading to a total cost 120 ms. Since communication dominates and imbalance contributes minimally, the cost model suggests increasing vector partitions to 2 and reducing dimension partitions to 3. After

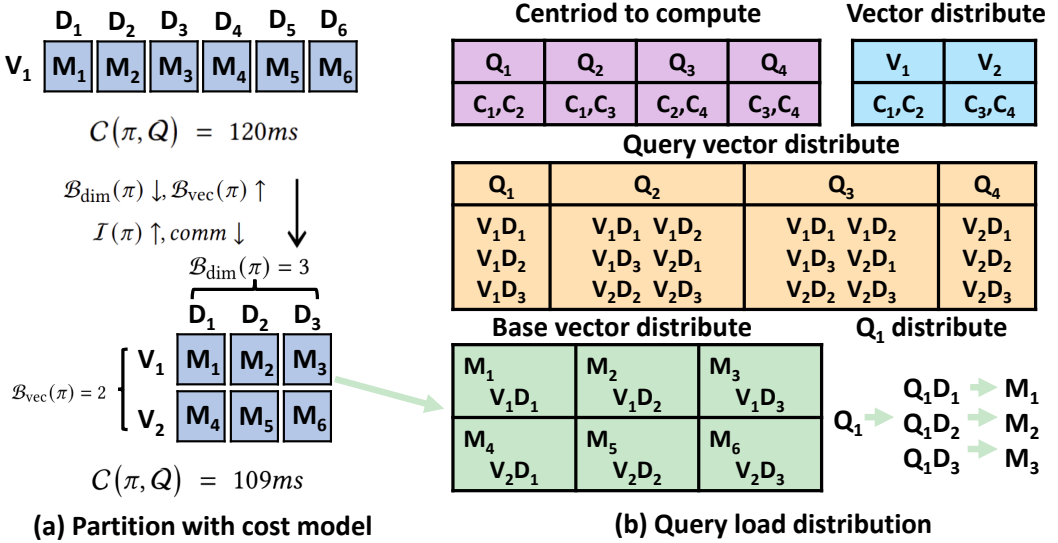


Fig. 4. Harmony's partition with cost model and query distribution.

adjustment, computation and communication costs become $c'_{comp} = 80$ ms and $c'_{comm} = 20$ ms, with $I(\pi') = 30$ and the same α . The total cost drops to 109 ms, confirming the new partitioning is more efficient under the model.

Partitioning strategy explanation. Harmony's partitioning scheme is derived based on the number of machines specified by the user. The scope of our strategy spans both vector-level and dimension-level granularity. The index is first partitioned into N dimension-based blocks, where each partition receives a subset of dimensions proportional to the compute capacity of its assigned machine. In homogeneous clusters, this typically corresponds to assigning each machine $\lceil d/N \rceil$ dimensions (i.e., $B_{vec}(\pi) = 1$) to maintain balanced computation. Partitioning is then refined through the cost model. Based on past queries, Harmony estimates the expected imbalance $I(\pi)$ of a given plan and adaptively increases the number of vector partitions $B_{vec}(\pi)$ to compensate, iteratively searching for a configuration that optimizes end-to-end cost.

Drawback of partition based on only vectors. While pure vector-level partitioning can also incorporate historical query patterns (e.g., assigning hot vectors more evenly), it lacks fine-grained adaptability. For instance, if past queries show that 20% of the vectors dominate 80% of computations, a vector-only scheme may distribute them evenly. However, if future queries deviate from this pattern, load imbalance may re-emerge. In contrast, Harmony's hybrid partitioning allows dimension-level splits within each vector group, ensuring that even under highly skewed workloads—e.g., when queries target only vectors from V_1 in Figure 4 (a), multiple machines (e.g., M_1 – M_3) can still contribute computation.

4.2.2 Query load distribution. Figure 4 (b) illustrates how queries are distributed. First, after the cost model analyzes the workload and finalizes a partition plan π , we obtain a grid of dimension splits $B_{dim}(\pi)$ and vector splits $B_{vec}(\pi)$. As shown in Figure 4 (a), dimension partitions $\{D_1, D_2, D_3\}$ and vector partitions $\{V_1, V_2\}$, result in blocks $\{V_1D_1, V_1D_2, \dots\}$. Each machine in the cluster (M_1 – M_6) holds one of these blocks completing the base vector distribution.

Second, after distributing the base vector, Harmony completes the query load distribution when the query vector is received, as shown in Figure 4 (b):

- (1) **Identify cluster centroids (purple table).** Like many cluster-based ANNS engines, Harmony first retrieves the relevant centroids on the client side to map each query q_i to the associated portion of the base vector.
- (2) **Map queries to vector-based blocks (blue table).** After retrieving the centroids, Harmony determines which vector partitions $\mathcal{B}_{\text{vec}}(\pi)$ each query should visit. For example, in Figure 4(b), $Q1$ is mapped to $V1$.
- (3) **Combine with dimension splits and map to machines (orange and green tables).** Once Harmony determines the vector partitions $\mathcal{B}_{\text{vec}}(\pi)$ for each query, it splits the queries based on dimension partitions $\mathcal{B}_{\text{dim}}(\pi)$. For example, $Q1$ mapped to $V1$ will be split into $Q1D1$, $Q1D2$, and $Q1D3$ corresponding to dimension partitions $\{D1, D2, D3\}$. These query chunks are routed to corresponding machines for partial distance computations. As shown in Figure 4(b), blocks like $V1D1$, $V1D2$, and $V1D3$ are assigned to $M1$, $M2$, and $M3$. This ensures $Q1D1$, $Q1D2$, and $Q1D3$ are processed in parallel on separate machines. By evenly distributing these workloads, Harmony avoids bottlenecks and fully utilizes cluster resources.

Advantages. Because the blocks in $\mathcal{B}_{\text{dim}}(\pi) \times \mathcal{B}_{\text{vec}}(\pi)$ are evenly distributed among machines, the four queries fully utilize all six nodes. No machine handles a disproportionately large portion of the dataset, resulting in balanced load and higher throughput.

Time complexity. The time complexity of Harmony's query distribution is determined by two factors: centroid assignment and query distribution across machines.

1. *Centroid Assignment (Computation overhead).* For Q query vectors with D dimensions and NC base centroids, finding the closest centroids costs $O(Q \times NC \times D^2)$, a standard operation in most ANNS engines without additional overhead specific to Harmony.

2. *Query Distribution (Communication overhead).* After centroid assignment, each query vector is distributed across machines for partial distance computations. Assume a query vector needs to interact with V machines. With Harmony, this number increases to $V \times B$, where B is the number of dimension-based splits $\mathcal{B}_{\text{dim}}(\pi)$. This finer-grained partitioning introduces additional communication overhead due to increased cross-machine interactions, which we analyze in detail in Section 4.3 and further validate experimentally in Section 6.2.

Space complexity. Harmony's space requirements are minimal as each base vector is stored on one machine, eliminating redundancy. The query set is typically small compared to base vectors, so storing and routing query splits incur negligible space costs. These space requirements ensure Harmony can scale effectively without significant memory consumption. We further validate this claim in Section 6.4.2.

4.3 Flexible Pipelined Execution Engine

Harmony leverages multiple machines to handle parts of a query's vectors or dimensions, enabling distributed pruning. However, this approach presents a challenge mentioned in Section 1: when each machine computes partial distances in isolation, it loses the ability to prune candidates based on results from other machines.

Solution. To resolve this, Harmony adopts a pipelined execution scheme, ensuring once a candidate vector or part of its dimension is deemed unpromising, no further machines need to process it.

Algorithm 1 illustrates how Harmony orchestrates vector-level and dimension-level distance computations in a pipeline. We first describe the algorithmic structure, then highlight the key lines.

- (1) **Prewarm stage (Lines 1-5).** $\text{PrewarmHeap}(Q, K)$ calculates initial distances for a subset of queries stored in the client node, including the centroid and a few other vectors. This populates a max-heap of size K , providing an early pruning threshold.

Algorithm 1: FLEXIBLEPIPELINEDPRUNING

```

1 Function PrewarmHeap (QuerySet  $Q$ , int  $K$ ):
    // Compute partial distances for a subset of queries to build an initial max-heap.
2     foreach  $q$  in  $Q\_subset$  do
3          $dist \leftarrow \text{ComputeDistance}(q, \text{randomVectors})$ 
4          $\text{heap\_insert}(q, dist, K)$ 
5     return heap
6 Function DimensionPipeline ( $q$ ,  $DSet$ ):
    // Process each dimension block in  $DSet$  sequentially.
7     foreach  $d$  in  $DSet$  do
8          $partialDist \leftarrow \text{ComputeDistance}(q, d)$ 
9         if  $partialDist > q.currentThreshold$  then
10             $\text{prune}(q)$ 
11            return
12         $\text{UpdatePruning}(q, partialDist)$ 
13 Function VectorPipeline (QueryBatch  $Q_v$ ,  $DSet$ ):
    // Process a batch of queries for a specific vector partition.
14     foreach  $q$  in  $Q_v$  do
15          $\text{DimensionPipeline}(q, DSet)$ 
16         if  $q.pruned == true$  then
17             continue
18          $\text{UpdatePruning}(q, finalDist)$ 
19 Function QUERYPIPELINE(QuerySet  $Q$ , VSetList  $VList$ , DSetList  $DList$ , int  $K$ ):
    // Stage 0: Prewarming
20      $\text{heap} \leftarrow \text{PrewarmHeap}(Q, K)$ 
    // Stage I: Vector-level pipeline
21     foreach  $vPart$  in  $VList$  do
22          $Q_v \leftarrow \text{filterQueries}(Q, vPart)$ 
23          $\text{VectorPipeline}(Q_v, DList)$ 

```

- (2) **Vector pipeline (Lines 13–18).** $\text{VectorPipeline}(Q_v, DList)$ iterates over each vector partition $\mathcal{V}$ in $VList$. Within a single vector-based search, it invokes the dimension pipeline (Line 15) to handle partial distance computations. After processing each vector partition, the pipeline updates pruning thresholds and discards queries no longer qualifying for the top- K set. This ensures each machine specializes in searching a particular vector partition, ensuring parallelism while respecting the global pruning condition.
- (3) **Dimension pipeline (Lines 6–12).** $\text{DimensionPipeline}(q, DSet)$ scans each dimension block in $\mathcal{B}_{\dim}(\pi)$ sequentially. For each block, the dimension pipeline computes a partial distance and compares it to the current pruning threshold. If the distance exceeds the threshold, the query is pruned immediately, skipping further dimensions. Otherwise, the pipeline updates the threshold to reflect the partial result, potentially lowering it for subsequent blocks. This fine-grained control allows unpromising queries to exit early, minimizing redundant computations across the cluster.

Complexity. We analyze Harmony’s complexity under balanced load. Let Q denote the number of query vectors, NB the number of base vectors (data points), and D the dimensionality of each vector, with distance computations costing up to $O(D^2)$ in the naive case. Additionally, let n_{list}

denote the total number of clusters (lists) and n_{probe} the number of probed clusters per query. These parameters define the computational and communication load across vector database.

Time Complexity. In a naive (centralized) scenario, one might compute $O(Q \times NB \times n_{\text{probe}}/n_{\text{list}} \times D)$ total operations. However, we typically distribute workload across $\mathcal{B}_{\text{vec}}(\pi) \times \mathcal{B}_{\text{dim}}(\pi)$ machines, where the total number of machines is specified by the user. Given this constraint, we select a suitable decomposition of $\mathcal{B}_{\text{vec}}(\pi)$ and $\mathcal{B}_{\text{dim}}(\pi)$ to balance workload and communication cost. Under balanced load, each machine handles:

$$\frac{Q \times (n_{\text{probe}}/n_{\text{list}}) \times NB \times D}{\mathcal{B}_{\text{vec}}(\pi) \times \mathcal{B}_{\text{dim}}(\pi)}.$$

Thus, the final per-machine cost scales as $O\left(\frac{Q \times n_{\text{list}} \times NB \times D}{n_{\text{probe}} \mathcal{B}_{\text{vec}}(\pi) \mathcal{B}_{\text{dim}}(\pi)}\right)$. Since the average computation cost per machine depends only on the total number of machines (i.e., $\mathcal{B}_{\text{vec}}(\pi) \mathcal{B}_{\text{dim}}(\pi)$), it does not vary significantly across different indexing strategies. Therefore, in our cost model, computation complexity is not the determining factor for choosing a particular partitioning scheme.

Communication Overhead. On average, each query interacts with $n_{\text{probe}} \times \frac{\mathcal{B}_{\text{vec}}(\pi)}{n_{\text{list}}}$ vector partitions. Each partition has $\mathcal{B}_{\text{dim}}(\pi)$ dimension blocks. The number of communication required is $\mathcal{B}_{\text{dim}}(\pi)$ times greater, but the amount of transfers remains constant. Consequently, the communication for query distribution grows proportionally to:

$$\left(\frac{NB}{\mathcal{B}_{\text{vec}}(\pi)}\right) \times \left(n_{\text{probe}} \times \frac{\mathcal{B}_{\text{vec}}(\pi)}{n_{\text{list}}}\right) \times D \times \mathcal{B}_{\text{dim}}(\pi).$$

Thus, the total communication cost is proportional to the number of dimension-based blocks $\mathcal{B}_{\text{dim}}(\pi)$, since each block contributes a separate portion of the distance result to be transmitted. As a result, partitioning by dimensions—while beneficial for balancing computation—can increase communication overhead. Therefore, Harmony must carefully balance the trade-off between load balancing and transmission cost when choosing the partitioning strategy.

Space Complexity. Each of the NB base vectors is split among $\mathcal{B}_{\text{vec}}(\pi)$ (vector) $\times \mathcal{B}_{\text{dim}}(\pi)$ (dimension) blocks. Thus, each block stores roughly

$$\frac{NB \times D}{\mathcal{B}_{\text{vec}}(\pi) \times \mathcal{B}_{\text{dim}}(\pi)}$$

elements, giving total storage of $NB \times D$ (no duplication). Intermediate query results are small compared to the raw dataset and can be ignored in practice. Consequently, the space cost remains $O(NB \times D)$, matching standard distributed systems without extra replication. Therefore, space complexity is not the determining factor for choosing a particular partitioning scheme.

Example. Figure 5 shows an example of pipeline and pruning strategy for Harmony’s vector and dimension granularity. Assuming query vectors $Q1 - Q6$ need to compute distances with base vector blocks $V1, V2$.

Vector-level pipeline and pruning. In the vector-level pipeline (Figure 5 (a)), queries are processed in batches corresponding to vector partitions ($\mathcal{B}_{\text{vec}}(\pi)$). For example, during Stage A, queries $Q1 - Q3$ are assigned to vector partition $V1$, while $Q4 - Q6$ are assigned to $V2$. Each batch completes its initial distance calculations and updates the global pruning condition by refining the max-heap threshold. This ensures that subsequent query batches ($Q4 - Q6$ and $V1$ in Stage B) benefit from a tighter pruning threshold, reducing unnecessary computations. Iteratively refining the heap after processing each batch, the vector-level pipeline achieves efficient pruning while maintaining parallelism across vector partitions.

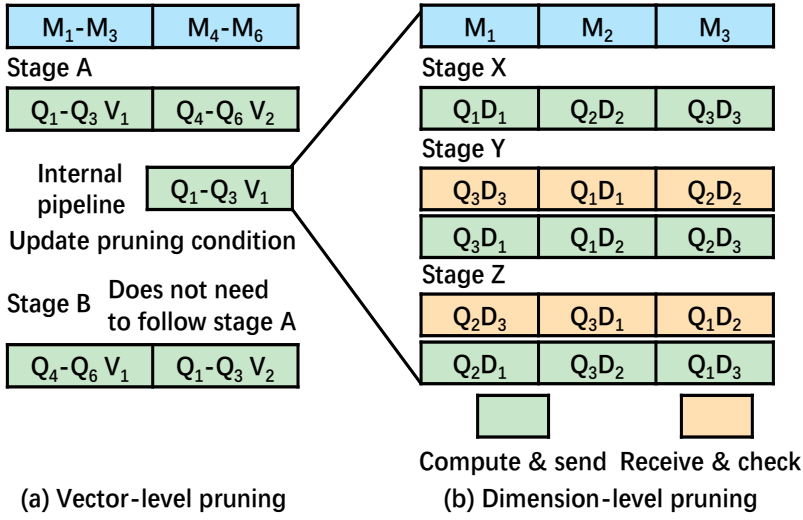


Fig. 5. Harmony's pipeline pruning and querying.

Dimension-Level Pipeline. In the dimension-level pipeline (Figure 5(b)), each vector's dimensions are distributed across machines, ensuring no multiple dimensions are processed simultaneously. This design guarantees dimension-wise computations proceed in a pipeline fashion, enabling effective pruning.

In Stage X, each machine begins processing the first assigned dimension block of the query. For instance, Q_1 begins with D_1 on M_1 , Q_2 starts with D_2 on M_2 , and Q_3 with D_3 on M_3 . After computing the partial distance for the first block, intermediate results are accumulated and compared against the global pruning threshold. If the accumulated distance exceeds the threshold, the query is pruned early, preventing further computations for the remaining dimensions.

In Stage Y, machines process the next dimension blocks while respecting the updated pruning conditions. For example, Q_1D_2 is sent to M_2 , Q_2D_3 to M_3 , and Q_3D_1 to M_1 . During this stage, the partial results computed in Stage X are reused and incremented with the newly calculated distances from the current dimension block. The pruning threshold is checked again, ensuring no unnecessary computation for unpromising queries.

Between stages, workers transmit partial results that are lightweight compared to the original vectors, incurring minimal bandwidth overhead. Furthermore, by asynchronously interleaving the processing of different queries—e.g., distributing dimensions of the same query across non-consecutive stages—the transmission of partial results no longer blocks the critical computation path. Each stage proceeds independently without stalling on others, effectively decoupling computation from communication. This design minimizes communication overhead in practice; we analyze the detailed cost breakdown of this optimization in Section 6.3.1.

This pipeline ensures that, at each stage, dimensions for a given query are separated across machines, eliminating overlap in computation. By leveraging the distribution of dimensions across nodes, Harmony avoids synchronization overhead typically required in dimension-partition implementations. This approach maximizes pruning while minimizing inter-machine communication and redundant calculations.

Advantages. By combining vector-level and dimension-level pipelines, Harmony achieves coarse-grained and fine-grained pruning in a single framework. Specifically, the vector-level pipeline updates a global threshold after each batch, preventing unnecessary computations for

subsequent ones. Meanwhile, the dimension-level pipeline processes dimension blocks in sequence, discarding queries whose partial distance exceeds the threshold. This dual-stage approach exploits the hierarchical distribution of data and queries, minimizing computation and communication overhead. Unlike previous methods that restrict dimension-based pruning to a single node [29], Harmony distributes dimensions across machines, eliminating interrupt-driven pruning checks and reducing synchronization overhead. This design also facilitates integration with hardware accelerations (e.g., SIMD), as the dimension-level pipeline decouples partial result transmissions from compute operations. Queries are pruned in real time, the workload is balanced across machines, and the system scales efficiently for large-scale ANNS.

Load Balancing Strategies. Although the initial distribution ensures balanced workloads across machines, pruning introduces imbalance over time. This occurs because dimensions processed later in the pipeline are more likely to be pruned, as discussed in Section 3.1. Resulting in reduced computation for some machines while others experience heavier loads. To address this, Harmony dynamically adjusts the execution order of dimensions to reduce load imbalance. For example, if the machine responsible for $D1$ (e.g., $M1$) becomes overloaded, subsequent queries (e.g., $Q2$) are reconfigured to process $D1$ last. As pruning is more effective in later stages, this adjustment significantly reduces the burden on overloaded machines.

In the example shown in Figure 5(b), $Q2$'s dimension $D1$ is deferred to the final stage, allowing earlier stages to prune unpromising queries and alleviate the load on $M1$. This adaptive scheduling ensures no machine becomes a bottleneck, maintaining balanced query execution and improving throughput.

This type of adaptive scheduling is not feasible under vector-level partitioning. Since vector-based schemes rely on a global Top- K pruning condition, all machines apply the same pruning threshold uniformly. As a result, their pruning rates are fixed and cannot be selectively adjusted across machines. In contrast, Harmony's dimension-level scheduling allows fine-grained control over pruning order, enabling flexible load redistribution and improved throughput.

4.4 Applicability of Harmony

Capability with different index methods. Harmony distributes the distance computation of a single query vector across multiple machines, making it well-suited for ANN algorithms with minimal inter-node dependencies during search. It naturally applies to **cluster-based** [14, 15, 17, 25, 37–39, 71, 78], **hash-based** [20, 34, 57, 63, 64], and **high-dimensional tree-based** [16, 48, 53, 61] methods, where candidates can be evaluated in parallel. Harmony is therefore free to partition and perform queries on these indexes at the vector and dimension levels. However, it is less effective for **graph-based indexes** [23, 24, 26, 27, 33, 51, 59, 60, 77], which require sequential path traversal, making parallelized distance computation inefficient.

Applicability to diverse scenarios. Harmony is particularly effective in scenarios with skewed workloads, where distributed load balancing is essential for maintaining system throughput. It also benefits tasks involving dimension-level pruning, where only a subset of features contributes significantly to the final distance, allowing early termination of computation. Moreover, because Harmony's core modification is applied at the fundamental level of distance computation, it remains orthogonal to the upper-layer query logic, making it compatible with hybrid query types such as scalar filters and range conditions.

5 Implementation

Harmony is implemented in C++20 with 6000 lines of code. It leverages modern C++ features, such as compile-time computation and zero-cost abstractions. It also employs OpenMPI for inter-node communication and Intel MKL for efficient distance computations.

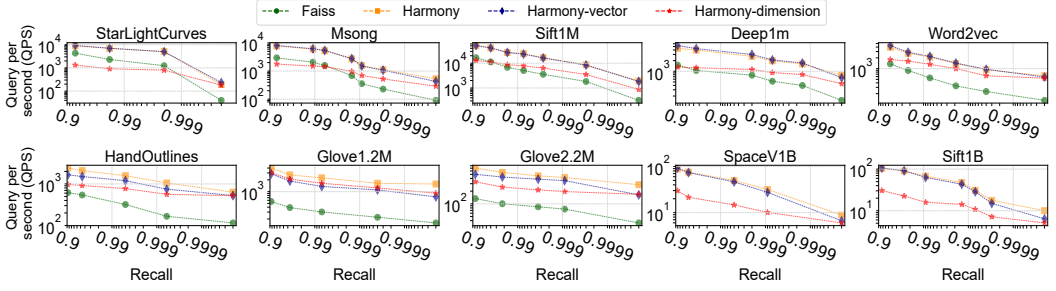


Fig. 6. Time-accuracy tradeoffs under balanced workloads (index variance = 0, query variance = 0, TopK=10).

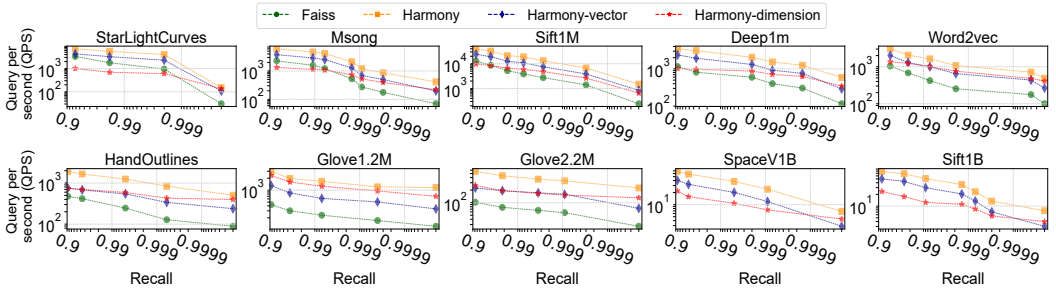


Fig. 7. Time-accuracy tradeoffs under skewed workloads (index variance = 500, query variance = 500, TopK=100).

To support fine-grained partitioning strategies such as dimension-level decomposition, Harmony extends the traditional master-worker architecture by introducing a lightweight synchronization mechanism for partial result exchange. During query execution, the master node maintains the global top- k heap and periodically updates pruning thresholds, which are broadcast to workers. Each worker performs local distance computations based on its assigned vector or dimension slice, and asynchronously communicates intermediate results using non-blocking MPI operations (MPI_Isend / MPI_Irecv). This design allows communication and computation to be effectively overlapped, reducing idle wait time and minimizing the synchronization cost introduced by dimension-level partitioning. Furthermore, the architecture remains modular—only the distance computation and coordination logic are extended, preserving compatibility with existing indexing methods and query planners. Within each node, distance calculations are parallelized using OpenMP, while Intel MKL accelerates core operations such as L2 distance and inner product.

Parameters. Harmony is built on top of Faiss and provides the same set of basic interfaces [*e.g.*, *nlist*, *nprobe*, *dim*], allowing users to adopt it without concern for internal implementation details. In addition, it introduces a few distributed-specific parameters to support partitioned execution in large-scale environments. These commonly used parameters are defined as follows:

- `-NMachine [nargs=1]`: Specify the number of nodes involved in the distributed index.
- `-Pruning_Configuration [nargs=0..1]`: Enables or disables vector and dimension-level pruning.
- `-Mode [Harmony, Harmony-vector, Harmony-dimension]`: Specify the mode.

6 Evaluation

6.1 Experiment Setup

Methodology. We primarily compare Harmony against Faiss [25], a state-of-the-art single-node ANNS engine, using its IVF and IVFPQ algorithms. To provide a comprehensive evaluation, we further include both distributed and single-node baselines: random-shard + HNSW [51] and random-shard + KNN; Auncel [79], a distributed method with accuracy guarantees; and several strong single-node alternatives, including HNSW with quantization and DiskANN [36]. We also evaluate two internal variants of Harmony: Harmony-vector, which uses vector-based partitioning, and Harmony-dimension, which partitions by feature dimensions. All methods adopt the same clustering algorithm and number of clusters to ensure fair comparison. After cluster construction, each approach applies its own distribution strategy. Harmony leverages a skew-aware cost model as described in Section 4.2.

Datasets. We use ten open-source datasets of varying sizes and dimensions, as shown in Table 2. These datasets are created by embedding various types of data (time series, audio, image, word vectors, and text) with different dimensions, providing both data and query vectors. They have been widely evaluated by many ANN systems [17, 77] and algorithms [26, 29, 51]. We expand the Star Light Curves and Hand Outlines datasets to ensure that they contain sufficient data for distributed processing while satisfying the base distribution.

Table 2. Dataset statistics.

Dataset	Size	Dim	Query Size	Data Type
Star Light Curves	823,600	1024	1,000	Time Series
Msong	992,272	420	1,000	Audio
Sift1M	1,000,000	128	10,000	Image
Deep1M	1,000,000	256	1,000	Image
Word2vec	1,000,000	300	1,000	Word Vectors
Hand Outlines	1,000,000	2709	370	Time Series
Glove1.2m	1,193,514	200	1,000	Text
Glove2.2m	2,196,017	300	1,000	Text
SpaceV1B	1,000,000,000	100	10,000	Text
Sift1B	1,000,000,000	128	10,000	Image

Platform. We conduct a standard performance evaluation of Harmony on a distributed system consisting of 20 nodes. Each node is equipped with an Intel(R) Xeon(R) Gold 6258R CPU, featuring 28 cores and 56 threads, and is connected to 256GB of global memory in a NUMA configuration. The nodes are interconnected via high-speed 100Gb/s networking, and the operating system used is CentOS Stream 8. Each node supports AVX-512, making it suitable for high-performance distributed workloads.

6.2 Overall Performance

6.2.1 QPS-recall trade-off. In this section, we analyze the trade-off between query per second (QPS) and recall when comparing Faiss with Harmony in different workloads. We test Harmony on four worker nodes and use Faiss as a baseline on a single node. For two large datasets, SpaceV1B and Sift1B, the single-node solution Faiss and the 4-node solution are unable to handle search requirements due to their size. We employ Harmony with 16 nodes for distributed search. To simulate realistic skewed workloads, we first perform a pre-query phase for each index using a workload with variance = 500 (as defined in Section 4.2.1), allowing each method to construct its

index based on this query distribution. We then evaluate all methods using test workloads with both variance = 0, TopK=10 and variance = 500, TopK=100 (and TopK=10 in Appendix A) to assess their performance under balanced and skewed conditions. The results are shown in Figures 6 and 7.

We have the following three findings. First, by combining with dimension-based pruning, all three distribution strategies in Harmony benefit from pruning optimization, achieving an average speedup of $4.12\times$ over Faiss, exceeding the theoretical $4\times$ ceiling under ideal conditions. Among them, Harmony further improves performance by pipelining pruning and communication, reaching an average speedup of $4.93\times$. However, Harmony-vector does not show a similarly significant improvement because dimension-level pruning introduces new load imbalance issues, as discussed in Challenge 3, which weakens the pruning benefits. Second, when the recall requirement is below 99%, the Harmony-Vector partitioning shows the optimal performance. This is because the speedup for Harmony-Dimension and Harmony partitioning is limited under lower precision, and the overhead from additional distance computations further impacts their performance. Third, in imbalanced workload scenarios, Harmony-Vector suffers from a significant performance drop—its QPS decreases by an average of 42% compared to balanced workloads. In contrast, Harmony and Harmony-Dimension maintain relatively stable QPS, demonstrating better robustness to load skew. This is because, beyond the intrinsic limitations of Harmony-vector itself, Harmony is able to dynamically rebalance workloads during execution while preserving pruning effectiveness. In contrast, Harmony-vector cannot translate its pruning rates into effective QPS improvement under skewed loads, as the final system throughput remains bottlenecked by the slowest group.

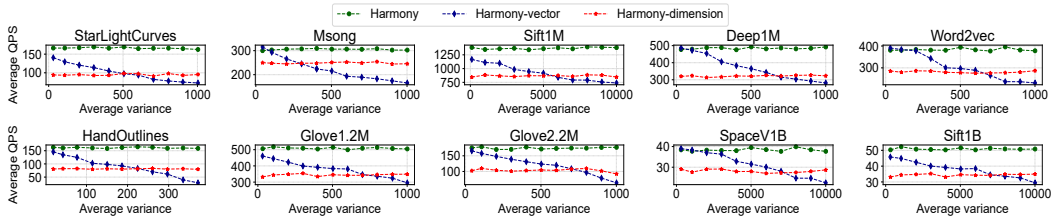


Fig. 8. Impact of workload distribution on query performance (Index variance = 500).

6.2.2 Search performance under skewed workloads. We conduct experiments on four distinct nodes, examining three different strategies of Harmony under varying levels of load imbalance. To simulate realistic query patterns, we first build a historical query workload with an average variance of 500 in per-machine load. Index partitions are then initialized based on this historical distribution. Specifically, Harmony-dimension partitions the index purely by dimensions, ignoring workload patterns. Harmony-vector distributes clusters by evenly assigning frequently accessed clusters across machines according to the observed query frequencies. Harmony applies the cost model described in Section 4.2.1 to determine a hybrid partitioning of vector and dimension blocks. During execution, queries are processed using the pipelined strategy described in Section 4.3. The final results are shown in Figure 8.

First, we observe that the traditional vector partition does not adapt well to load imbalance scenarios. As the load becomes more imbalanced, the average QPS decreases 56% on average. Second, Harmony and Harmony-dimension partitioning (dim) both handle imbalance more favorably, showing no clear performance degradation during load skew changes. Although the dimension-based approach is more stable due to its absolute partitioning principle, the hybrid approach in Harmony (balancing dim and vector partitioning adaptively) remains advantageous. Even when the load is extremely imbalanced, Harmony outperforms the dimension-based partitioning by 91%,

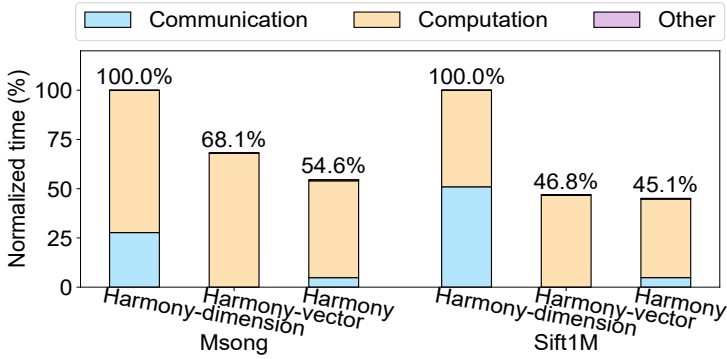


Fig. 9. The contribution of the three optimization techniques to Harmony’s query throughput.

demonstrating the advantage of our cost model. Third, upon further inspection of these datasets, we notice that Harmony retains its advantage consistently for larger-sized vectors or higher feature dimensions. As complexity grows and distribution skew worsens, our adaptive strategy in Harmony continues to leverage the synergy between dimension-split and vector-split, thus providing robust performance across varying degrees of imbalance.

6.3 Performance Ablation Study

6.3.1 Time breakdown. We analyzed the time spent in various stages of the query process, including data communication, computation, and other overhead. The results are shown in Figure 9.

We have the following three key findings. First, except for Harmony-vector, both Harmony and Harmony-dimension incur communication overhead. Specifically, Harmony-dimension has a higher communication overhead due to more dimension slicing. This is evident as the communication time is larger in Harmony-dimension compared to Harmony, which has less dimension partitioning. Second, despite the similarity in computation overhead between Harmony-dimension and Harmony-vector, Harmony has a lower computational overhead compared to the other two. This is because Harmony introduces a pruning module, which will be analyzed further in Section 6.3.3, where we discuss pruning ratios for each module. Third, the main overheads for all three modules are concentrated in communication and computation, and the communication overhead is independent of dimensions, meaning that communication overhead is significantly lower than computation overhead in datasets with larger dimensions. For example, in the 128-dimensional Sift1M dataset, communication overhead is much higher compared to the 420-dimensional Msong dataset.

6.3.2 Contribution of optimization techniques. To understand the contribution of each component in Harmony, we measure the speedup provided by its key features: balanced load, pipeline and synchronous execution, and pruning in 4 nodes. By isolating these components, we assess their effects on system performance. Experimental results are shown in Figure 10.

We have three main findings. First, on average, each optimization technique provides a performance improvement, with balanced load yielding a $1.75\times$ increase, pipeline and asynchronous execution achieving a $1.25\times$ improvement, and pruning offering a $1.51\times$ enhancement in throughput. Second, the performance gain comes from pruning, aligning with the findings in Section 3.1. This demonstrates that pruning is an impactful optimization technique for improving Harmony’s query throughput. Third, for the Sift1M dataset, the performance improvement from balanced load and pipeline execution is less pronounced. This is due to the relatively uniform distribution of the load,

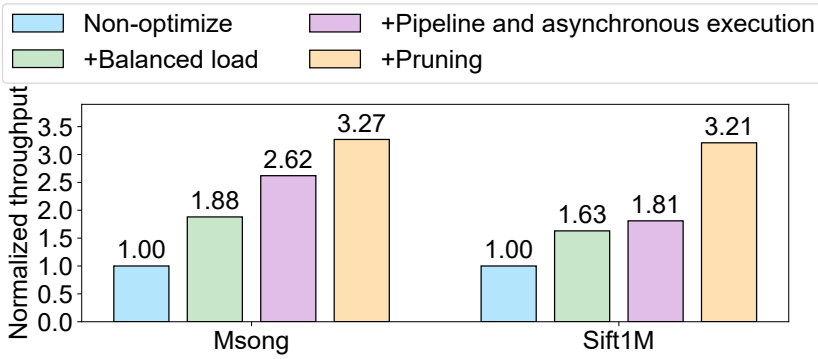


Fig. 10. The contribution of the three optimization techniques to Harmony's query throughput.

where the benefit of load balancing and asynchronous execution is somewhat diminished. However, pruning still leads to a clear throughput improvement, indicating its robustness in optimizing query performance regardless of data distribution.

6.3.3 Pruning ratio breakdown. We measure the pruning strategy discussed in Section 4.3 to evaluate its effectiveness in reducing computational overhead. In this experiment, we perform a dimensional split of size 4 and apply the pruning strategy to assess its impact on query performance. The results are shown in Table 3.

Table 3. Average pruning ratio comparison across four nodes.

Dataset	First Slice (%)	Second Slice (%)	Third Slice (%)	Fourth Slice (%)	Average Pruning Ratio (%)
Msong	0.00	43.14	76.06	95.29	53.87
Glove1.2m	0.00	1.54	30.71	86.66	29.73
Word2vec	0.00	24.85	53.77	83.66	40.32
Deep1M	0.00	7.67	66.09	97.36	42.03
Sift1m	0.00	41.76	85.04	98.40	57.05
Star	0.00	81.24	95.23	99.05	69.14
Glove2.2m	0.00	5.14	30.70	81.18	29.76
Hand	0.00	63.54	91.62	98.10	63.83

We have the following three observations. First, later slices exhibit better pruning rates. Specifically, the average pruning ratio for the second slice is 33.61%, for the third slice is 66.15%, and for the fourth slice is 92.33%. This is because subsequent query slices benefit from the results of earlier slices. As discussed in Section 3.1, later slices can achieve better pruning thresholds due to the accumulation of query results. Second, pruning rates vary significantly across different datasets. For instance, in the Glove1.2M dataset, the third slice achieves only a 30.71% pruning rate, while in the Star dataset, it reaches 95.23%. This variance is mainly due to the differences in dataset distributions, where some datasets are more easily pruned than others. Third, the pruning rate in the final slice consistently exceeds 80%. This indicates that the vast majority of distance computations do not require the use of the last dimension, which aligns with our analysis in Section 3.1. This highlights that most vectors are pruned by the time the final slice is reached.

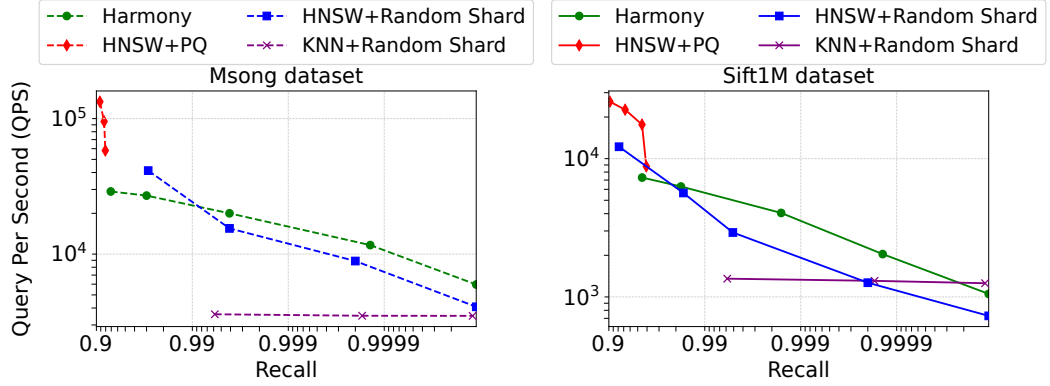


Fig. 11. Performance comparison with different baselines.

6.4 Index Build Experiments

6.4.1 Index build time. We measure the time and space overhead of constructing indexes using Harmony and Faiss in Appendix B.

6.4.2 Index space overhead. In this experiment, we compare the space overhead of Harmony’s index with Faiss and baseline configurations. Efficient use of memory is critical for large-scale ANNS systems, and this experiment quantifies the storage efficiency of Harmony’s hybrid partitioning and pruning mechanisms. The results are shown in Table 4.

Table 4. Index memory comparison.

Dataset	Faiss	Harmony vector	Harmony dimension	Harmony
StarLightCurves	3.2GB	788MB	815MB	798MB
Msong	1.6GB	411MB	418MB	413MB
Sift1M	497MB	126MB	131MB	128MB
Deep1M	986MB	245MB	253MB	250MB
Word2Vec	1.2GB	258MB	295MB	279MB
HandOutlines	6.1GB	1.50GB	1.54GB	1.51GB
Glove1.2M	921MB	227MB	238MB	233MB
Glove2.2M	2.5GB	660MB	697MB	686MB

We identify the following three key points. First, the index overhead is proportional to both the dataset size and its dimensions. For example, the space used by Msong is 3.2 times larger than that of Sift1M, which aligns with their respective dataset sizes (dimension $\times$ dataset size). Second, all three partitioning schemes occupy about 1/4 of the space of Faiss (single-machine index), suggesting that distributed indexing does not require significant additional space overhead. Third, both Harmony and Harmony-dimension incur some additional space overhead, but this overhead is minimal, representing only about 2% of the original space size.

6.5 Additional Baselines

We further compare Harmony against a broader set of baselines, including two representative graph-based indexing schemes for large-scale data: (1) HNSW with random sharding across four nodes, and (2) single-node HNSW combined with Product Quantization (HNSW+PQ). In addition,

we include (3) KNN without indexing, executed across four nodes with random sharding. These configurations provide a fair basis for comparison with Harmony. The experimental results are shown in Figure 11.

Random Shard + HNSW. We randomly partition the dataset across multiple nodes and build a local HNSW graph on each machine independently using the Faiss HNSW index implementation. In the four-node setting, we observe that this approach is on average 53% slower than Harmony and Harmony-Vector under recall = 99.99%. The performance gap arises because HNSW requires each machine to independently traverse its own entry points, leading to redundant path exploration and increased total computation across nodes. However, at lower recall levels, HNSW with random sharding can outperform Harmony in terms of QPS, as its graph structure allows for fast approximate navigation without scanning large candidate lists.

HNSW with quantization. To reduce memory usage on single nodes, we also evaluate HNSW combined with product quantization (HNSW-PQ), using Faiss's HNSWPQ index to build compressed graphs. However, as shown in Figure 11, HNSW-PQ fails to achieve high query throughput (QPS) compared to memory-resident approaches. Even though it can reach over 98% recall at TopK = 100 on several datasets, the QPS remains low due to the additional decoding overhead and less efficient distance computations introduced by quantization. Moreover, at smaller TopK values such as TopK = 10, HNSW-PQ is less effective—it cannot maintain high recall under tight accuracy constraints, with recall plateauing around 93% in our tests. This confirms that quantization introduces trade-offs that are particularly pronounced under high-precision, low-TopK settings.

Random Shard + KNN. To further evaluate the effectiveness of Harmony under ultra-high precision requirements, we conduct a comparison against brute-force KNN search. In this experiment, we directly search the vector dataset without any index structures. Specifically, we measure and compare the QPS–Recall performance in the regime where recall exceeds 0.99. The results demonstrate that brute-force KNN can outperform Harmony in extremely high-precision scenarios on low-dimensional datasets such as Sift1M. However, as dimensionality increases—as seen in the Msong dataset—the QPS of KNN drops significantly. This highlights the necessity of building indexing structures in high-dimensional vector search, where exhaustive methods quickly become inefficient.

Comparison with disk-based solutions. Disk-based solutions offer another approach to scaling large-scale vector search by storing indexes on secondary storage rather than fitting them entirely in memory. To provide a comprehensive comparison, we also evaluate DiskANN [36], a representative disk-optimized approximate nearest neighbor (ANN) system. DiskANN builds a navigable small-world graph and dynamically fetches vector data from disk during query processing to minimize memory usage. In our evaluation, we configure DiskANN to cache only 1/100 of the vectors in memory, with the remaining data accessed from disk during search. Under this setup, we find that DiskANN achieves a QPS that is 12.4% that of Harmony when targeting 99% recall, highlighting the significant throughput advantage of memory-resident approaches in high-accuracy regimes.

Comparison with Auncel. Auncel [79] focuses on handling distributed vector queries with low search load under specific precision requirements. It uses a fixed partitioning strategy similar to Harmony-vector, distributing the query load across the system. However, Auncel's performance in load balancing is not optimal under skewed workloads, which is similar to the behavior of Harmony-vector. Experimental results show that under a workload with query variance = 500, Auncel is 22% slower than Harmony.

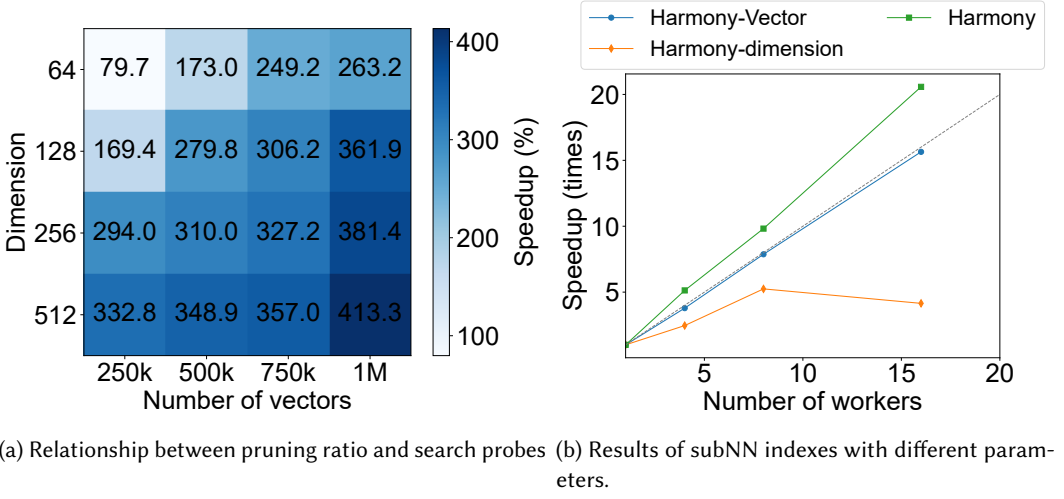


Fig. 12. Impact of pruning effectiveness and indexing parameters on search performance

6.6 Additional Discussion

Impact of dimensions and dataset size on harmony performance. We construct datasets that follow a Gaussian distribution, with dimensions ranging from 64 to 512 and sizes varying from 250K to 1M, using four nodes. The speedup achieved by Harmony is shown in Figure 12 (a).

We have three key findings. First, as both the dataset size and dimension increase, performance improves. Specifically, for each doubling of the dimension, the speedup increases by 26.8%, and for each doubling of the dataset size, the speedup increases by 25.9%. Second, in large datasets (1M, 512 dimensions), Harmony shows a speedup greater than the number of machines. This is due to the introduction of pruning techniques, which reduce computational overhead. Third, Harmony’s performance is suboptimal for smaller datasets, which is attributed to the increased communication overhead that dominates the overall performance in such cases.

Scalability. To validate the scalability of Harmony, we perform experiments testing its performance across different machine configurations: 4, 8, and 16 nodes. The speedup achieved by Harmony is shown in Figure 12 (b).

We have three key findings. First, the group-based partitioning method consistently demonstrates a speedup greater than the number of machines, owing to the pruning design, which effectively reduces computational overhead. Second, the vector-based partitioning achieves performance close to the number of workers, as it does not introduce additional overhead and scales linearly with the number of workers. Third, the dimension-based partitioning method exhibits an increase in performance as the number of workers grows, followed by a decline. This occurs because as the number of dimensions increases, communication overhead grows, which eventually reduces the overall performance gain.

Peak memory usage. To evaluate memory efficiency during query execution, we measure the peak memory consumption of Harmony and compare it to baseline partition methods in 4 nodes in Appendix C.

Comparison with index with PQ. To further evaluate the compatibility of Harmony with compressed indexes, we test the integration of Product Quantization (PQ) into both Harmony and Harmony-Vector. The experimental results are consistent with the observations in Section 6.5: PQ provides notable speed advantages at lower recall targets, but it fails to achieve recall higher than

97% on most datasets due to quantization loss. Additionally, we observe that when combined with PQ, the relative speedup of Harmony over Harmony-Vector under skewed workloads is reduced, averaging around 13%. This is because the time spent on distance computations—which is the main contributor to load imbalance—is significantly reduced after applying PQ. As a result, the performance impact of load skew becomes less pronounced in PQ-accelerated settings.

Cost model evaluation. To evaluate the effectiveness of our cost model, we test how well Harmony and Harmony-vector perform relative to a perfect index under varying workload skew. Specifically, we first construct both indexes based on a historical query set with variance = 500, allowing them to organize their partitions accordingly. We then run evaluation queries with two levels of skew: variance = 500 and variance = 1000. The results show that under query variance = 500, Harmony achieves performance within 3% of a perfectly balanced index, whereas Harmony-vector suffers a 21% degradation. When the query variance increases to 1000, Harmony remains robust, with only a 5% performance drop, while Harmony-vector's performance drops by over 40%. These results demonstrate that Harmony's cost model effectively adapts to skewed workloads, significantly improving robustness compared to purely vector-based partitioning.

Combination with PDX. PDX [41] eliminates the SIMD performance bottleneck in dimension-pruning workloads by enabling efficient vectorized processing across dimensions. This aligns naturally with Harmony's block-based vector storage strategy, as both approaches benefit from reduced branch mispredictions and improved parallelism. We estimate that integrating PDX optimizations into Harmony and Harmony-vector would yield an average 1.4× speedup at 99% recall on our low-dimensional datasets. This observation aligns with the improvements reported in the original PDX paper [41].

7 Conclusion

This paper introduces Harmony, an efficient solution for scaling Approximate Nearest Neighbor Search (ANNS) in cluster-based distributed vector databases. By combining dimension-based and vector-based partition, Harmony ensures balanced load distribution and minimizes communication costs. Experimental results show that Harmony outperforms existing systems like Faiss, achieving 4.93× throughput on average in 4 nodes and 58% performance improvement over traditional distribution for skewed loads.

Acknowledgments

This work is supported by the National Natural Science Foundation of China (No. 62461146205 and 62322213), Beijing Natural Science Foundation (L242017), Beijing Nova Program (No. 20230484397 and 20220484137), and the Strategic Priority Research Program of Chinese Academy of Sciences, Grant No.XDB0500103. Qian Xu, Feng Zhang, Chengxi Li, Zheng Chen, and Xiaoyong Du are with the Key Laboratory of Data Engineering and Knowledge Engineering (MOE), and School of Information, Renmin University of China. Feng Zhang is the corresponding author of this paper.

References

- [1] 2017. JanusGraph. <https://janusgraph.org/>. Accessed: 2025-01-08.
- [2] 2024. UCI Machine Learning Repository. <https://archive.ics.uci.edu/>.
- [3] 2024. YouTube. <https://www.youtube.com/>. <https://www.youtube.com>
- [4] 2025. Apache Drill. <https://drill.apache.org/>.
- [5] 2025. CockroachDB. <https://www.cockroachlabs.com/product/cockroachdb/>.
- [6] 2025. Integrating Presto with Spark SQL. <https://prestodb.io/docs/current/connector/spark.html>.
- [7] 2025. Neo4j. <https://neo4j.com/>.
- [8] 2025. Pinecone. <http://pinecone.io>. <http://pinecone.io>
- [9] 2025. Presto: Distributed SQL Query Engine for Big Data. <https://prestodb.io/>.

- [10] 2025. Qdrant. <http://qdrant.tech>. <http://qdrant.tech>
- [11] 2025. TigerGraph. <https://www.tigergraph.com/>.
- [12] 2025. Vald. <http://vald.vdaas.org>. <http://vald.vdaas.org>
- [13] 2025. Vespa. <https://vespa.ai/>. <https://vespa.ai>
- [14] Artem Babenko, Victor Lempitsky, B.Hari Babu, N.Subhash Chandra, and T.V. Gopal. 2014. The inverted multi-index. *IEEE transactions on pattern analysis and machine intelligence* 37, 6 (2014), 1247–1260.
- [15] Dmitry Baranchuk, Artem Babenko, and Yuri Malkov. 2018. Revisiting the inverted indices for billion-scale approximate nearest neighbors. In *Proceedings of the European Conference on Computer Vision (ECCV)*. 202–216,.
- [16] Jon Louis Bentley. 1975. Multidimensional binary search trees used for associative searching. *Communications of the ACM* 18, 9 (1975), 509–517.
- [17] Qi Chen, Bing Zhao, Haidong Wang, Mingqin Li, Chuanjie Liu, Zengzhong Li, Mao Yang, and Jingdong Wang. 2021. Spann: Highly-efficient billion-scale approximate nearest neighborhood search. In *Advances in Neural Information Processing Systems*, M. Ranzato, A. Beygelzimer, Y. Dauphin, P.S. Liang, and J.Wortman Vaughan (Eds.). Vol. 34. Curran Associates, Inc, 5199–5212.
- [18] Kenneth L. Clarkson. [n. d.]. An algorithm for approximate closest-point queries. In *Proceedings of the tenth annual symposium on Computational geometry*. 160–164, 1994.
- [19] James C. Corbett, Jeffrey Dean, Michael Epstein, Andrew Fikes, Christopher Frost, J. J. Furman, Sanjay Ghemawat, Andrey Gubarev, Christian Heiser, Peter Hochschild, and et al. 2012. Spanner: Google’s Globally-Distributed Database. In *Proceedings of the 10th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. 251–264.
- [20] Mayur Datar, Nicole Immorlica, Piotr Indyk, and Va-hab S. Mirrokni. 2004. Locality-sensitive hashing scheme based on p-stable distributions. In *Proceedings of the Twentieth Annual Symposium on Computational Geometry, SCG '04*. 253–262,.
- [21] Jeffrey Dean and Luiz André Barroso. 2013. The tail at scale. *Commun. ACM* 56, 2 (2013), 74–80.
- [22] Jeffrey Dean and Sanjay Ghemawat. 2008. MapReduce: Simplified data processing on large clusters. In *Communications of the ACM*, Vol. 51. ACM New York, NY, USA, 107–113.
- [23] B.N. Delaunay. 1934. Sur la sphère vide. *Bull. Acad. Sci. URSS* 6 (1934), 793–800.
- [24] Wei Dong, Moses Charikar, and Kai Li. 2011. Efficient k-nearest neighbor graph construction for generic similarity measures. In *Proceedings of the 20th International Conference on World Wide Web, WWW2011*. Association for Computing Machinery, Hyderabad, India, 577–586. doi:10.1145/1963405.1963487
- [25] Facebook. 2020. Faiss. <https://github.com/facebookresearch/faiss>. Accessed: 2024-10-15.
- [26] Cong Fu, Chao Xiang, Changxu Wang, and Deng Cai. 2019. Fast approximate nearest neighbor search with the navigating spreading-out graph. *Proc. VLDB Endow.* 12, 5 (jan 2019), 461–474. doi:10.14778/3303753.3303754
- [27] K.Ruben Gabriel and Robert R. Sokal. 1969. A new statistical approach to geographic variation analysis. *Systematic zoology* 18, 3 (1969), 259–278.
- [28] Jianyang Gao, Yutong Gou, Yuexuan Xu, Yongyi Yang, Cheng Long, and Raymond Chi-Wing Wong. 2025. Practical and Asymptotically Optimal Quantization of High-Dimensional Vectors in Euclidean Space for Approximate Nearest Neighbor Search. *Proc. ACM Manag. Data* 3, 3, Article 202 (June 2025), 26 pages. doi:10.1145/3725413
- [29] Jianyang Gao and Cheng Long. 2023. High-Dimensional Approximate Nearest Neighbor Search: with Reliable and Efficient Distance Comparison Operations. *Proc. ACM Manag. Data* 1, 2, Article 137 (jun 2023), 27 pages. doi:10.1145/3589282
- [30] Jianyang Gao and Cheng Long. 2024. RaBitQ: Quantizing High-Dimensional Vectors with a Theoretical Error Bound for Approximate Nearest Neighbor Search. *Proc. ACM Manag. Data* 2, 3, Article 167 (may 2024), 27 pages. doi:10.1145/3654970
- [31] Wayne D. Gray and Deborah A. Boehm-Davis. 2000. Milliseconds matter: An introduction to microstrategies and to their use in describing and predicting interactive behavior. *Journal of experimental psychology: applied* 6, 4 (2000).
- [32] Rentong Guo, Xiaofan Luan, Long Xiang, Xiao Yan, Xiaomeng Yi, Jigao Luo, Qianya Cheng, Weizhi Xu, Jiarui Luo, Frank Liu, et al. 2022. Manu: a cloud native vector database management system. *arXiv preprint arXiv:2206.13843* (2022).
- [33] Kiana Hajebi, Yasin Abbasi-Yadkori, Hossein Shahbazi, and Hong Zhang. 2011. Fast Approximate Nearest-Neighbor Search with k-Nearest Neighbor Graph. In *IJCAI 2011, Proceedings of the 22nd International Joint Conference on Artificial Intelligence*. Barcelona, Catalonia, Spain. doi:10.5591/978-1-57735-516-8/IJCAI11-222 AAAI Press, 1312–1317..
- [34] P. Jain, B. Kulis, and K. Grauman. 2008-06. Fast image search for learned metrics. In *2008 IEEE Conference on Computer Vision and Pattern Recognition*. 1–8,.
- [35] Virajith Jalaparti, Peter Bodik, Srikanth Kandula, Ishai Menache, Mikhail Rybalkin, and Chenyu Yan. 2013. Speeding up distributed request-response workflows. *ACM SIG-COMM Computer Communication Review* 43, 4 (2013), 219–230.

- [36] Suhas Jayaram Subramanya, Fnu Devvrit, Harsha Vardhan Simhadri, Ravishankar Krishnawamy, and Rohan Kadekodi. 2019. DiskANN: Fast Accurate Billion-point Nearest Neighbor Search on a Single Node. In *Advances in Neural Information Processing Systems*, H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett (Eds.), Vol. 32. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2019/file/09853c7fb1d3f8ee67a61b6bf4a7f8e6-Paper.pdf
- [37] Herve Jegou, Matthijs Douze, and Cordelia Schmid. 2010. Product quantization for nearest neighbor search. *IEEE transactions on pattern analysis and machine intelligence* 33, 1 (2010), 117–128.
- [38] Hervé Jégou, Romain Tavenard, Matthijs Douze, and Laurent Amsaleg. 2011. Searching in one billion vectors: rerank with source coding. In *2011 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 861–864.
- [39] Yannis Kalantidis and Yannis Avrithis. 2014. Locally optimized product quantization for approximate nearest neighbor search. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2321–2328.
- [40] Noam Koenigstein, Parikshit Ram, and Yuval Shavitt. 2012. Efficient retrieval of recommendations in a matrix factorization framework. In *Proceedings of the 21st ACM international conference on Information and knowledge management*. 535–544.
- [41] Leonardo Kuffo, Elena Krippner, and Peter Boncz. 2025. PDX: A Data Layout for Vector Similarity Search. *Proc. ACM Manag. Data* 3, 3, Article 196 (June 2025), 26 pages. doi:10.1145/3725333
- [42] Brian Kulis and Kristen Grauman. 2009. Kernelized locality-sensitive hashing for scalable image search. In *Computer Vision, 2009 IEEE 12th International Conference on*. IEEE, Elsevier Science Publishers B, V., NLD, 2130–2137.
- [43] Hui Li, Tsz Nam Chan, Man Lung Yiu, and Nikos Mamoulis. 2017. Fexipro: fast and exact inner product retrieval in recommender systems. In *Proceedings of the 2017 ACM International Conference on Management of Data*. 835–850.
- [44] Jie Li, Haifeng Liu, Chuanghua Gui, Jianyu Chen, Zhenyuan Ni, Ning Wang, and Yuan Chen. 2018. The Design and Implementation of a Real Time Visual Search System on JD E-Commerce Platform. In *Proceedings of the 19th International Middleware Conference Industry (Rennes, France)*. Association for Computing Machinery, New York, NY, USA, 9–16. doi:10.1145/3284028.328403
- [45] J. Li, M. Zhang, K. Niu, Y. Zhang, Y. Ke, and X. Yang. 2025. High-Security HEVC Video Steganography Method Using the Motion Vector Prediction Index and Motion Vector Difference. *Tsinghua Science and Technology* 30, 2 (April 2025), 813–829. doi:10.26599/TST.2024.9010016
- [46] Sen Li, Fuyu Lv, Taiwei Jin, Guli Lin, Keping Yang, Xiaoyi Zeng, XiaoMing Wu, and Qianli Ma. 2021. Embedding-Based Product Retrieval in Taobao Search. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining (KDD '21)*. Association for Computing Machinery, New York, NY, USA, 3181–3189. doi:10.1145/3447548.3467101
- [47] Defu Lian, Haoyu Wang, Zheng Liu, Jianxun Lian, Enhong Chen, and Xing Xie. 2020. Lightrec: A memory and search-efficient recommender system. In *Proceedings of The Web Conference 2020*. 695–705.
- [48] Ting Liu, Andrew W. Moore, Alexander Gray, and Ke Yang. December 13–18, 2004. An investigation of practical approximate nearest neighbor algorithms. In *Advances in Neural Information Processing Systems 17 [Neural Information Processing Systems, NIPS 2004]*. Vancouver, British Columbia, Canada, 825–832.
- [49] Yaoyang Liu, Zhen Zheng, Feng Zhang, Jincheng Feng, Yiyang Fu, Jidong Zhai, Bingsheng He, Xiao Zhang, and Xiaoyong Du. 2025. A Comprehensive Taxonomy of Prompt Engineering Techniques for Large Language Models. *Frontiers of Computer Science* (2025). doi:10.1007/s11704-025-50058-z
- [50] Fangzhou Alec Lu. 2024. Towards software-defined FPGA acceleration for big data analytics. (2024).
- [51] Yu A. Malkov and D. A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Trans. Pattern Anal. Mach. Intell.* 42, 4 (apr 2020), 824–836. doi:10.1109/TPAMI.2018.2889473
- [52] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. 2013. Efficient Estimation of Word Representations in Vector Space. arXiv:1301.3781 [cs.CL].
- [53] Marius Muja and David G. Lowe. 2014. Scalable Nearest Neighbour Algorithms for High Dimensional Data. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 36, 11 (2014), 2227–2240.
- [54] Rajesh Nishtala, Hans Fugal, Steven Grimm, Marc Kwiatkowski, Herman Lee, Harry C. Li, Ryan McElroy, Mike Paleczny, Daniel Peek, and Paul Saab. 2013. Scaling memcache at facebook. In *10th USENIX Symposium on Networked Systems Design and Implementation (NSDI 13)*. 385–398.
- [55] Zhen Peng, Minjia Zhang, Kai Li, Ruoming Jin, and Bin Ren. 2023. iQAN: Fast and Accurate Vector Search with Efficient Intra-Query Parallelism on Multi-Core Architectures. In *Proceedings of the 28th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming, PPoPP 2023, Montreal, QC, Canada, 25 February 2023 - 1 March 2023*, Maryam Mehri Dehnavi, Milind Kulkarni, and Sriram Krishnamoorthy (Eds.). ACM, 313–328. doi:10.1145/3572848.3577527
- [56] Jeffrey Pennington, Richard Socher, and Christopher Manning. 2014. Glove: Global Vectors for Word Representation. *EMNLP* 14 (2014), 1532–1543. doi:10.3115/v1/D14-1162

- [57] Jie Ren, Minjia Zhang, and Dong Li. 2020. HM-ANN: Efficient BillionPoint Nearest Neighbor Search on Heterogeneous Memory. *Proceedings of the 34th International Conference on Neural Information Processing Systems* 895 (2020), 20.
- [58] Jeff Shute and et al. 2013. F1: A Distributed SQL Database That Scales. In *Proceedings of the VLDB Endowment*, Vol. 6. 1068–1079.
- [59] Godfried T. Toussaint. 1980. The relative neighbourhood graph of a finite planar set. *Pattern recognition* 12, 4 (1980), 261–268.
- [60] Jing Wang, Jingdong Wang, Gang Zeng, Zhuowen Tu, Rui Gan, and Shipeng Li. 2012. Scalable k-nn graph construction for visual descriptors. In *Computer Vision and Pattern Recognition (CVPR), 2012 IEEE Conference on*. IEEE, IEEE Computer Society, USA.
- [61] Jingdong Wang, Naiyan Wang, You Jia, Jian Li, Gang Zeng, Hongbin Zha, and Xian Sheng Hua. 2014. Trinary-projection trees for approximate nearest neighbor search. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 36, 2 (2014), 388–403.
- [62] Jianguo Wang, Xiaomeng Yi, Rentong Guo, Hai Jin, Peng Xu, Shengjun Li, Xiangyu Wang, Xiangzhou Guo, Chengming Li, Xiaohai Xu, Kun Yu, Yuxing Yuan, Yinghao Zou, Jiquan Long, Yudong Cai, Zhenxiang Li, Zhifeng Zhang, Yihua Mo, Jun Gu, Ruiyi Jiang, Yi Wei, and Charles Xie. 2021. Milvus: A Purpose-Built Vector Data Management System. In *Proceedings of the 2021 International Conference on Management of Data (Virtual Event, China) (SIGMOD '21)*. Association for Computing Machinery, New York, NY, USA, 2614–2627. doi:10.1145/3448016.3457550
- [63] Jingdong Wang, Ting Zhang, Jingkuan Song, Nicu Sebe, and Heng Tao Shen. 2018. A survey on learning to hash. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 40, 4 (2018), 769–790.
- [64] Yair Weiss, Antonio Torralba, and Rob Fergus. 2009. Spectral hashing. In *Advances in neural information processing systems*. 1753–1760.
- [65] J. Wen, S. Pei, C. Yan, K. Xie, and W. Liang. 2025. Extensible Bloom Filters: Adaptive Strategies for Scalability and Efficiency in Network and Distributed Systems to Handle Increased Data. *Tsinghua Science and Technology* 30, 4 (August 2025), 1846–1864. doi:10.26599/TST.2024.9010160
- [66] Tom White. 2012. *Hadoop: The Definitive Guide* (3rd ed.). O'Reilly Media, Inc.
- [67] Q. Wu, X. Cui, M. Lu, P. Yang, and P. Wu. 2024. A GNSS Anti-Spoofing Technique Based on the Spatial Distribution Characteristic of the Residual Vectors. *Tsinghua Science and Technology* 29, 2 (April 2024), 457–468. doi:10.26599/TST.2023.9010017
- [68] Shitao Xiao, Zheng Liu, Weihao Han, Jianjin Zhang, Yingxia Shao, Defu Lian, Chaozhuo Li, Hao Sun, Denvy Deng, and Liangjie Zhang. 2022. Progressively optimized bi-granular document representation for scalable embedding based retrieval. In *Proceedings of the ACM Web Conference 2022*. 286–296.
- [69] Qian Xu, Juan Yang, Feng Zhang, Zheng Chen, Jiawei Guan, Kang Chen, Ju Fan, Youren Shen, Ke Yang, Yu Zhang, and Xiaoyong Du. 2024. Improving Graph Compression for Efficient Resource-Constrained Graph Analytics. *Proc. VLDB Endow.* 17, 9 (May 2024), 2212–2226. doi:10.14778/3665844.3665852
- [70] Qian Xu, Juan Yang, Feng Zhang, Junda Pan, Kang Chen, Youren Shen, Amelie Chi Zhou, and Xiaoyong Du. 2025. Tribase: A Vector Data Query Engine for Reliable and Lossless Pruning Compression using Triangle Inequalities. *Proc. ACM Manag. Data* 3, 1, Article 82 (Feb. 2025), 28 pages. doi:10.1145/3709743
- [71] Yuming Xu, Hengyu Liang, Jin Li, Shuotao Xu, Qi Chen, Qianxi Zhang, Cheng Li, Ziyue Yang, Fan Yang, Yuqing Yang, Peng Cheng, and Mao Yang. 2023. SPFresh: Incremental In-Place Update for Billion-Scale Vector Search. In *Proceedings of the 29th Symposium on Operating Systems Principles, SOSP 2023, Koblenz, Germany, October 23-26, 2023*, Jason Flinn, Margo I. Seltzer, Peter Druschel, Antoine Kaufmann, and Jonathan Mace (Eds.). ACM, 545–561. doi:10.1145/3600006.3613166
- [72] Matei Zaharia, Mosharaf Chowdhury, Michael J Franklin, Scott Shenker, and Ion Stoica. 2010. Spark: Cluster computing with working sets. In *Proceedings of the 2nd USENIX conference on Hot topics in cloud computing (HotCloud)*, Vol. 10. 10–10.
- [73] Shulin Zeng, Zhenhua Zhu, Jun Liu, Haoyu Zhang, Guohao Dai, Zixuan Zhou, Shuangchen Li, Xuefei Ning, Yuan Xie, Huazhong Yang, and Yu Wang. 2023. DF-GAS: A Distributed FPGA-as-a-Service Architecture towards Billion-Scale Graph-based Approximate Nearest Neighbor Search. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture (Toronto, ON, Canada) (MICRO '23)*. Association for Computing Machinery, New York, NY, USA, 283–296. doi:10.1145/3613424.3614292
- [74] Feng Zhang, Jidong Zhai, Xipeng Shen, Onur Mutlu, and Xiaoyong Du. 2022. POCLib: A High-Performance Framework for Enabling Near Orthogonal Processing on Compression. *IEEE Transactions on Parallel and Distributed Systems* 33, 2 (2022), 459–475. doi:10.1109/TPDS.2021.3093234
- [75] Feng Zhang, Jidong Zhai, Xipeng Shen, Dalin Wang, Zheng Chen, Onur Mutlu, Wenguang Chen, and Xiaoyong Du. 2021. TADOC: Text analytics directly on compression. *The VLDB Journal* 30, 2 (2021), 163–188.
- [76] Feng Zhang, Chenyang Zhang, Jiawei Guan, Qiangjun Zhou, Kuangyu Chen, Xiao Zhang, Bingsheng He, Jidong Zhai, and Xiaoyong Du. 2025. Breaking the Edge: Enabling Efficient Neural Network Inference on Integrated Edge Devices.

- IEEE Transactions on Cloud Computing* (2025). doi:10.1109/TCC.2025.3559346
- [77] Qianxi Zhang, Shuotao Xu, Qi Chen, Guoxin Sui, Jiadong Xie, Zhizhen Cai, Yaoqi Chen, Yinxuan He, Yuqing Yang, Fan Yang, Mao Yang, and Lidong Zhou. 2023. VBASE: Unifying Online Vector Similarity Search and Relational Queries via Relaxed Monotonicity. In *17th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2023, Boston, MA, USA, July 10-12, 2023*, Roxana Geambasu and Ed Nightingale (Eds.). USENIX Association, 377–395. <https://www.usenix.org/conference/osdi23/presentation/zhang-qianxi>
- [78] Ting Zhang, Chao Du, and Jingdong Wang. 2014. Composite quantization for approximate nearest neighbor search. In *Proceedings of the 31th International Conference on Machine Learning (ICML, Vol. 32)*. 838–846.
- [79] Zili Zhang, Chao Jin, Linpeng Tang, Xuanzhe Liu, and Xin Jin. 2023. Fast, Approximate Vector Queries on Very Large Unstructured Datasets. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. USENIX Association, Boston, MA, 995–1011. <https://www.usenix.org/conference/nsdi23/presentation/zhang-zili>

Received January 2025; revised April 2025; accepted May 2025